Learn-By-Building Guide:
Multimodal Healthcare Diagnostic & CDSS

A concept-by-concept, code-by-code walkthrough for someone who knows text/tabular data science but is new to image data — written so you can build every part yourself.

How to use this document

This is not a copy-paste solution file — it is a teaching path. Each module explains a concept in plain language first, then shows the exact function(s) you'd call to apply it, with a small runnable example. Work through the modules in order; each one assumes only what the previous ones taught you. Wherever you see a "Try it yourself" prompt, stop and actually run the code on a few samples before moving on — that's where the real learning happens.

A quick word on how to read the theory sections: every new concept here is introduced by first asking "what problem does this solve?" before saying "here is the technical name for it." That order matters — it's far easier to remember what a convolution is once you already understand why raw pixels are a bad direct input to a plain neural network, than to memorize the definition of convolution first and hope the reason shows up later. So resist the urge to skim straight to the code blocks; the paragraphs directly above each one exist to make the code feel obvious rather than magical.

> *You already know data science on text/tabular data — regression, classification, train/test splits, pandas, sklearn. This guide leans on that constantly: almost every new image/text/deep-learning idea below is introduced by first describing its closest tabular-data-science equivalent, and only then explaining exactly how and why it differs. If a paragraph feels slow, it's because it's deliberately over-explaining a genuinely new idea rather than assuming you'll infer it from a one-line definition.*

Learning Path Overview

Before diving into any single module, it helps to see the whole shape of the journey — where you're starting from, where you're headed, and which specific new idea each stop adds. The table below is a map, not a summary to memorize: keep coming back to it as you finish each module so you always know why the next one exists and what gap it's closing.

Module	What you'll learn	New concept vs. text/tabular DS
0. Setup	Environment, libraries, folder structure	—
1. Images as data	How a picture becomes numbers a model can use	Tensors have 3 dimensions (channel, height, width) instead of 1 row of features
2. Seeing & inspecting images	Loading, displaying, resizing, augmenting	PIL / OpenCV instead of pandas
3. CNNs & transfer learning	Why convolution works, using a pretrained vision model	Feature extraction is learned spatially, not by hand-engineered columns
4. Building the image classifier	Full training loop for the vision-only baseline	DataLoader/Dataset pattern replaces a single X_train, y_train array
5. Clinical text with transformers	Tokenization, embeddings, pretrained BERT-family models	Text becomes a sequence of vectors, not TF-IDF columns
6. Fusing image + text	Combining two encoders into one model	Multi-branch neural network instead of a single feature table
7. Explainability	Grad-CAM for images, attention for text	Visual/heatmap explanations instead of feature-importance bar charts

8. Evaluation for medical AI	AUROC, calibration, per-class metrics	Same sklearn tools you already know, applied carefully to imbalanced labels
9. Serving your model	Wrapping it in an API + simple UI	FastAPI instead of a notebook cell
10. Glossary & cheat-sheet	Every function used, in one place	Quick reference while you build

Notice the pattern running down that middle column: almost every module is really just answering "how do I do the tabular-data-science thing I already know how to do, but for images or text instead of spreadsheet columns?" Keeping that framing in mind turns this from ten unrelated new topics into one repeated exercise: take a familiar concept, then translate it.

Module 0 — Environment Setup

Before writing a single line of modeling code, you need a clean, isolated Python environment. This matters more than it might seem: deep-learning libraries like PyTorch and Transformers move fast and have very specific version dependencies on each other and on your GPU drivers. If you install them into the same environment you use for other projects, you risk a library upgrade silently breaking something unrelated months from now. A virtual environment is simply a self-contained folder holding its own Python interpreter and its own copies of installed packages, completely separate from your system-wide Python — the same idea as a fresh, disposable sandbox for this one project.

```
python -m venv venv
source venv/bin/activate        # on Windows: venv\Scripts\activate

pip install torch torchvision torchaudio
pip install transformers datasets
pip install pandas numpy scikit-learn matplotlib pillow opencv-python
pip install grad-cam                # explainability
pip install fastapi uvicorn python-multipart   # serving
```

A short explanation of what each of these libraries is actually for, since you'll be importing from all of them repeatedly throughout this guide:

Library	Role in this project
torch / torchvision / torchaudio	PyTorch itself, plus torchvision's ready-made vision models, datasets, and image transforms
transformers / datasets	HuggingFace's library for pretrained language models (like Bio_ClinicalBERT) and dataset utilities
pandas / numpy	The tabular and numeric-array tools you already know, still used constantly for labels, metrics, and array math
scikit-learn	Evaluation metrics (AUROC, F1, etc.) and simple utilities like train/test splitting
matplotlib	Displaying images, training curves, and Grad-CAM heatmaps
pillow (PIL) / opencv-python	Two different libraries for opening, resizing, and manipulating image files
grad-cam	A ready-made implementation of the Grad-CAM explainability technique used in Module 7
fastapi / uvicorn	For turning your finished model into a small web API in Module 9

Next, set up a predictable folder structure. Doing this now — before you have any files to put in it — saves you from a messy, hard-to-navigate project later, the same way you'd plan a sensible directory layout before starting a large tabular-data project with multiple notebooks and datasets.

```
project/
```

```
data/raw/            <- unzip the Kaggle dataset here
data/processed/
notebooks/
src/
```

Module 1 — Images as Data (the core mental shift)

This module contains, in a sense, the single most important idea in the entire guide — everything from Module 2 onward is really just a consequence of it. So take it slowly.

1.1 What a "row" means for a picture

In every tabular project you've done, one training example is one row of a spreadsheet: a fixed list of numbers, each with a clear meaning (age, blood pressure, income, and so on). A model like logistic regression or XGBoost takes that row of numbers and produces a prediction.

An image is still, underneath everything, just a grid of numbers — but structured very differently. Instead of one row of independent columns, you have a 3-dimensional grid: width (how many pixels across), height (how many pixels down), and channels (how many numbers describe each individual pixel's colour). A standard colour photo has 3 channels — Red, Green, and Blue — because mixing different intensities of those three colours of light can reproduce almost any colour the human eye can see. A grayscale medical X-ray typically has just 1 channel, since each pixel is only a single brightness value (how light or dark that point is), with no colour information at all.

Concretely: a 224-pixel-wide, 224-pixel-tall colour image is a grid of $224 \times 224 \times 3 = 150{,}528$ individual numbers, each one typically ranging from 0 (no intensity / black) to 255 (full intensity), before any further processing. That is still "just numbers" in the sense you're used to — the difference is that these numbers have a spatial relationship to each other (a pixel is strongly related to its neighbours) that a plain row of independent tabular columns never has. A patient's age and their cholesterol level don't have a "neighbour" relationship the way the pixel at position (10, 10) and the pixel right next to it at (10, 11) do — those two pixels are almost always similar in brightness, because real-world objects have continuous surfaces, not random noise from pixel to pixel.

1.2 Why this spatial structure changes everything downstream

That one fact — nearby pixels are related, and that relationship carries meaning (edges, textures, shapes) — is the entire reason a specialised kind of neural network (the Convolutional Neural Network, or CNN, covered in Module 3) exists at all. If you tried to feed all 150,528 numbers of an image directly into a plain fully-connected layer (the kind you'd use for tabular data), the model would treat pixel (0,0) and pixel (223,223) as no more or less related than any other two input numbers, throwing away exactly the spatial information that makes an image recognisable in the first place. It would also need an enormous number of weights (150,528 inputs connected to even a modest hidden layer already means tens of millions of parameters), making the model slow to train and extremely prone to overfitting on a medical dataset of only a few thousand images.

Convolutional layers solve this by looking at small local neighbourhoods of pixels at a time (say, a 3×3 patch) and sliding that same small "lens" across the whole image, reusing the same small set of weights everywhere. That single design choice is what lets CNNs learn to recognise edges, then textures, then shapes, then whole anatomical structures — layer by layer — using a manageable number of parameters. You'll see exactly how this works with real code in Module 3; for now, the important takeaway is purely conceptual:

> 💡 *Everything downstream — convolution, pooling, CNNs — exists because feeding raw pixels into a plain dense layer (like you would with tabular features) would be hugely inefficient and would ignore the fact that nearby pixels are related. That single idea motivates the entire next module.*

Module 2 — Loading, Viewing, and Preparing Images

Now that you understand conceptually what an image is made of, this module is about the practical, hands-on side: how do you actually open an image file in Python, look at it, and turn it into the exact numeric format a

model expects? Think of this as the image-data equivalent of pd.read_csv() followed by basic pandas exploration (.head(), .describe(), a histogram or two) before you ever touch a model.

2.1 Load and view an image

The Python Imaging Library (PIL, imported as the Pillow package but used under the name PIL) is the standard tool for opening image files — PNG, JPEG, and similar formats — and represents an opened image as a special Image object you can inspect, transform, and eventually convert into a numeric array. Just like you'd call .head() on a new DataFrame before doing anything else, the very first thing to do with any new image dataset is to actually look at a few images with your own eyes, so you have an intuitive sense of what the data looks like before you write a single line of preprocessing code.

```
from PIL import Image
import matplotlib.pyplot as plt

img = Image.open('data/raw/images/1_IM-0001-4001.png')
img = img.convert('L')          # 'L' = grayscale, 'RGB' = colour
print(img.size)                 # (width, height)

plt.imshow(img, cmap='gray')
plt.axis('off')
plt.show()
```

A couple of details worth understanding rather than just running: .convert('L') forces the image into single-channel grayscale mode — 'L' stands for "Luminance", a photography term for perceived brightness. Chest X-rays are naturally grayscale, but the file itself might still be stored with 3 identical RGB channels depending on how it was exported, so explicitly converting keeps you in control of exactly how many channels you're working with rather than guessing. img.size returns (width, height) — note the order, since this is the opposite of how array shapes are usually written (which is typically height-first, then width — a genuinely common source of confusing bugs later, so it's worth remembering now).

2.2 Turn it into a tensor a model can use

A PIL Image object is great for viewing and light editing, but PyTorch models don't operate on Image objects directly — they operate on tensors, PyTorch's version of a numpy array (with the added ability to track gradients and run on a GPU, both covered later). So every image needs to pass through a short pipeline of steps before it can be fed into a model: resizing to a fixed size, converting to a tensor, and normalizing its pixel values. torchvision.transforms.Compose lets you chain these preprocessing steps together into a single reusable object — this is the direct image-data equivalent of building an sklearn Pipeline or ColumnTransformer that bundles scaling, encoding, and imputation into one object you call once.

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.Resize((224, 224)),        # every image must be the same size
    transforms.ToTensor(),                 # PIL image -> tensor, scales 0-255 to 0-1
    transforms.Normalize(                  # match the stats the pretrained model expects
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

img_rgb = img.convert('RGB')   # pretrained models expect 3 channels
tensor = transform(img_rgb)    # shape: [3, 224, 224]
```

Walking through each step and the reasoning behind it: Resize((224, 224)) forces every image, regardless of its original dimensions, to the exact same width and height. This is necessary because the network architecture you'll load in Module 3 has a fixed number of weights wired up to expect a specific input size — much like a tabular

model needs every row to have the same number of columns in the same order. ToTensor() does two things at once: it converts the PIL Image into a PyTorch tensor, and it rescales pixel values from the original 0–255 integer range down to a 0.0–1.0 floating-point range, which is a far more numerically stable range for a neural network to learn from (large raw pixel values can make training unstable, similar to why you'd scale tabular features before feeding them into a model sensitive to feature magnitude, like logistic regression or a neural network, though tree-based models like XGBoost don't need this).

Normalize() then shifts and rescales each channel again, this time using specific mean and standard deviation values. Why those exact numbers ([0.485, 0.456, 0.406] and [0.229, 0.224, 0.225])? They are the average pixel statistics computed across the entire ImageNet dataset — the 1.4-million-image dataset that the pretrained model you'll use in Module 3 was originally trained on. Because you are about to reuse that pretrained model's learned weights (transfer learning, explained fully in Module 3), your new input images need to be statistically similar in scale and distribution to what the model saw during its original training, or its early layers will misinterpret the input. It's a subtle but important detail: think of it as making sure you're speaking the same numeric "dialect" the pretrained model already understands.

> 💡 *Resize to a fixed size because CNNs (specifically the fully-connected layers at the end) expect a fixed input shape — same reason a tabular model needs a fixed number of columns for every row. The specific mean/std values above are the ImageNet averages; you use them because you're about to reuse a model that was trained on ImageNet, so your inputs need to look statistically similar to what it saw during training.*

2.3 Data augmentation (only during training)

Medical imaging datasets are almost always small relative to the millions of images used to train general-purpose vision models — the IU X-ray dataset has a few thousand images, not millions. With a small dataset, a large neural network can effectively "memorize" the training images rather than learning generalizable patterns — this is overfitting, the same phenomenon you already know from tabular modelling, just showing up here for a different underlying reason (too few examples relative to model capacity, rather than too many correlated features).

Data augmentation combats this by creating many slightly different versions of each training image on the fly — a small random crop, a horizontal flip, a slight brightness change — so the model sees a wider variety of "versions" of the same underlying image across different training epochs, and is discouraged from memorizing exact pixel arrangements. This plays a very similar role to techniques you may already know from tabular data, like adding small amounts of noise to numeric features, or oversampling/resampling techniques such as SMOTE for imbalanced classes — the shared underlying goal in both cases is making the model robust to minor variations rather than overly reliant on exact training examples.

```
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
])
```

Notice this augmented pipeline resizes to a slightly larger 256×256 first, then randomly crops down to the final 224×224 — this randomizes exactly which part of the image ends up as the model's input on each training pass, rather than always showing the exact same framing. RandomHorizontalFlip is generally safe for natural photographs, but worth a moment of clinical thought for medical images specifically: flipping a chest X-ray left-to-right could be reasonable (still an anatomically plausible chest), but flipping it top-to-bottom almost certainly would not be (no real X-ray looks like that), so augmentation choices should always be sanity-checked against what's actually medically plausible for your specific imaging modality, not applied blindly from a generic image-augmentation tutorial.

> 💡 *Try it yourself: load 3–4 X-ray images, apply train_transform to each 5 times, and plt.imshow() the results side by side. You should see slightly shifted/flipped/brightness-varied versions of the same image.*

Module 3 — CNNs and Transfer Learning

3.1 What a convolution actually does (intuition, no heavy math)

Recall from Module 1 that plain fully-connected layers are a poor fit for images because they ignore spatial structure and require an unmanageable number of weights. A convolutional layer fixes both problems with one elegant trick: instead of connecting every input pixel to every output neuron, it defines a small filter (also called a kernel) — typically a tiny grid of learnable numbers, like 3×3 — and slides that same filter across every position in the image, computing a weighted sum of the pixels currently "under" it at each position.

Because the exact same small filter is reused at every position across the whole image, the number of learnable parameters stays tiny (a 3×3 filter has only 9 weights, regardless of whether the image is 224×224 or 2,000×2,000), and — crucially — a pattern the filter learns to detect in the top-left corner of an image will be detected equally well if it appears in the bottom-right corner, because it's the same filter being applied everywhere. This property is called translation invariance, and it's a very natural fit for medical images: a particular kind of lung opacity should be recognisable as that same finding regardless of exactly where in the chest cavity it happens to appear.

A single convolutional layer typically uses many different filters at once (not just one), each free to learn to detect a different simple pattern — one filter might learn to respond strongly to vertical edges, another to horizontal edges, another to a particular texture. The outputs of many such filters are then stacked into what's called a feature map. A deep CNN stacks many convolutional layers on top of each other: the very first layer's filters tend to learn simple, generic things like edges and colour blobs; layers a bit deeper combine those edges into small textures and simple shapes; layers deeper still combine those shapes into complex, task-specific structures — for a chest X-ray model, this might eventually mean something as specific as "the rounded, hazy pattern characteristic of a lung opacity." This layer-by-layer building-up of increasingly abstract, increasingly task-specific patterns is often called the network learning its own feature hierarchy.

If you're used to the tabular-data workflow of manually engineering features (say, computing a ratio of two lab values because you believe it's clinically meaningful), the CNN's job is conceptually the same goal — turning raw input into useful, informative features — except the network discovers on its own, purely from data and the training signal, which visual patterns are actually useful for the task, rather than a human deciding this in advance. That's a genuinely powerful shift, but it also means a CNN typically needs a reasonably large amount of training data to discover good features on its own — which leads directly into the next idea, transfer learning.

3.2 Why transfer learning instead of training from scratch

Training a CNN completely from scratch — starting every filter's weights at random values and letting the network gradually learn useful patterns purely from your own dataset — generally requires a very large amount of labelled training data, often millions of images, to reliably learn good general-purpose filters (especially the simple, low-level ones like edge detectors, which are useful for almost any visual task and don't really need to be re-learned each time). A medical imaging dataset of a few thousand images is nowhere near enough to train a large CNN from scratch without severe overfitting.

Transfer learning sidesteps this problem by reusing a model that has already been trained on a large, general-purpose image dataset — most commonly ImageNet, a benchmark dataset of about 1.4 million natural photographs across 1,000 everyday object categories (dogs, cars, furniture, and so on). The key insight is that the early and middle layers of such a model — the ones learning edges, textures, and simple shapes — turn out to transfer remarkably well to completely different domains, including medical imaging, even though a chest X-ray looks nothing like a photo of a dog. Only the final, most task-specific layers (the ones that combine those shapes into "this looks like a golden retriever") really need to be replaced and retrained for your new task.

This is conceptually very similar to an idea you may already be familiar with from NLP: rather than training word embeddings completely from scratch on a small text dataset, you'd typically start from pretrained embeddings (like GloVe or word2vec) or a pretrained language model, because those already encode a huge amount of general linguistic knowledge learned from a far larger corpus than you could realistically collect yourself. The vision equivalent, ImageNet-pretrained CNNs, plays exactly the same role for image tasks — you're borrowing already-learned general visual knowledge and adapting only the final layers to your specific, smaller, specialised dataset.

3.3 Loading a pretrained vision model and adapting it

In practice, this means downloading a well-known CNN architecture together with its already-trained ImageNet weights, then surgically replacing its final classification layer — the one that currently outputs a score for each of the 1,000 ImageNet object categories — with a new, freshly-initialized layer sized for however many findings your own dataset has. DenseNet121 is used in the example below because it's a well-established, frequently-cited choice specifically for chest X-ray classification in the medical imaging literature (it's the backbone used by the well-known CheXNet model), but ResNet50 is an equally reasonable and very commonly used alternative.

```
import torch
import torch.nn as nn
from torchvision import models

model = models.densenet121(weights='IMAGENET1K_V1')

# The final layer currently outputs 1000 ImageNet classes.
# Replace it with a layer sized for YOUR number of findings.
num_findings = 14   # e.g. the 14 CheXpert-style labels
model.classifier = nn.Linear(model.classifier.in_features, num_findings)

# Optional: freeze early layers so only the new head (and maybe the
# last block) gets updated at first -- trains faster, needs less data.
for name, param in model.features.named_parameters():
    if 'denseblock4' not in name:   # keep the last block trainable
        param.requires_grad = False
```

Two things in this snippet deserve a slower explanation. First, model.classifier.in_features reads off how many numbers the layer right before the final classifier produces (DenseNet121 happens to produce 1024), so that the replacement layer nn.Linear(...) has the correct input size to connect to the rest of the network — you're keeping the entire "body" of the pretrained network intact and only swapping out its final "head."

Second, the freezing loop at the bottom sets requires_grad = False on most of the network's parameters, which tells PyTorch not to update those weights during training at all — effectively locking in the pretrained knowledge in the early layers, and only allowing the last block (denseblock4) plus your new classifier head to actually learn from your medical data. This is a very common and practical strategy when you have a comparatively small dataset: it trains faster (fewer parameters to update means less computation per step), needs less data to avoid overfitting (fewer free parameters means less opportunity to memorize), and generally still performs well, because — as discussed above — the earlier, frozen layers were already learning broadly useful, transferable visual features. Once you have a working baseline, unfreezing more layers and fine-tuning the whole network with a smaller learning rate is a reasonable next experiment.

> 💡 *model.classifier is DenseNet's name for its final layer; different architectures name it differently (model.fc for ResNet, model.head for some ViTs). Print the model (print(model)) to see the actual layer names before you edit one.*

Module 4 — Building the Image-Only Classifier (full training loop)

4.1 Dataset and DataLoader — the image version of X_train

With tabular data, once you've done your preprocessing, you typically end up with the entire dataset sitting in memory as one or two arrays — X_train and y_train — which you pass directly to model.fit(). Image datasets

almost never fit comfortably in memory this way: thousands of high-resolution images, each already a few hundred thousand numbers before you even get to a batch of them, would require an enormous amount of RAM to hold all at once as a single decoded array.

PyTorch solves this with a two-part pattern: a Dataset object, which doesn't hold any images in memory itself but instead knows how to produce one single sample on demand (given an index, go find that one image file on disk, open it, apply the transform, and return it alongside its label); and a DataLoader, which wraps around a Dataset and handles the practical mechanics of turning many individual samples into shuffled batches, optionally loading multiple images in parallel via separate worker processes for speed. You define the Dataset class once; PyTorch and your DataLoader handle the batching, shuffling, and iteration logic for you.

```
from torch.utils.data import Dataset, DataLoader
import pandas as pd
from PIL import Image

class ChestXrayDataset(Dataset):
    def __init__(self, csv_path, image_dir, transform):
        self.df = pd.read_csv(csv_path)     # columns: image_filename, label_0..label_13
        self.image_dir = image_dir
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(f"{self.image_dir}/{row['image_filename']}").convert('RGB')
        img = self.transform(img)
        labels = torch.tensor(row[1:].values.astype('float32'))
        return img, labels

train_ds = ChestXrayDataset('data/processed/train.csv', 'data/raw/images', train_transform)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2)
```

Every custom PyTorch Dataset needs exactly these three methods, and understanding each one's job makes the pattern reusable for any future dataset you build, image or otherwise: __init__ runs once, when you first create the dataset object — here it just reads a small CSV of filenames and labels into memory (this CSV is tiny compared to the actual images, so holding it in memory is no problem), and remembers the folder path and the transform pipeline to apply later. __len__ simply reports how many samples exist in total, which the DataLoader needs to know in order to plan out batches and to know when it has gone through a full epoch. __getitem__ is where the real, per-sample work happens: given an index, it looks up that row, opens the corresponding image file from disk at that moment (not ahead of time — this is what keeps memory usage manageable), applies your preprocessing pipeline from Module 2, and returns the resulting tensor alongside its label as a tensor too.

On the DataLoader line, batch_size=32 means the model will see 32 images at a time per training step, rather than one at a time (mirroring mini-batch gradient descent, which you may already be familiar with from training neural networks on tabular data) or the entire dataset at once (which usually wouldn't fit in memory or GPU memory for images). shuffle=True re-randomizes the order the samples are drawn in at the start of every epoch, which helps prevent the model from learning spurious patterns related to the order data happens to be stored in. num_workers=2 tells PyTorch to use 2 separate background processes to load and preprocess images in parallel while the GPU is busy computing on the previous batch, which can meaningfully speed up training since image loading and decoding from disk is often slower than the actual matrix math.

4.2 The training loop

In many tabular-modelling libraries, calling model.fit(X, y) hides the entire training process behind one function call. PyTorch deliberately does the opposite: you write the training loop yourself, explicitly, step by step. This is more verbose, but it gives you full visibility and control over exactly what happens at each step — which

becomes essential once you move to more complex setups like the multimodal fusion model in Module 6, where a single .fit() call wouldn't even know how to handle two different types of input at once.

```
import torch.optim as optim

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()   # multi-label -> one sigmoid per finding
optimizer = optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(10):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()          # clear old gradients
        outputs = model(images)        # forward pass
        loss = criterion(outputs, labels)
        loss.backward()                # compute gradients
        optimizer.step()               # update weights

        running_loss += loss.item()
    print(f'Epoch {epoch+1}: avg loss = {running_loss/len(train_loader):.4f}')
```

It's worth understanding this loop as a repeating four-beat rhythm, because you will write essentially this exact same rhythm again in Module 6 for the fused model, and in any future PyTorch project: first, clear out any leftover gradients from the previous step (zero_grad) — without this, PyTorch would keep adding new gradients on top of old ones, which is almost never what you want. Second, run the current batch of images forward through the network to get its current predictions (the "forward pass" — this is exactly the same forward() method you defined, or that torchvision defined, for the model). Third, compare those predictions to the true labels using a loss function to get a single number summarizing how wrong the model currently is, then call .backward() on that loss, which triggers backpropagation — PyTorch automatically works out, for every single weight in the entire network, how much that weight contributed to the current error. Fourth and finally, the optimizer actually uses those computed gradients to nudge every weight slightly in the direction that should reduce the loss (optimizer.step()). Repeating this four-beat cycle over and over, batch after batch, epoch after epoch, is the entire mechanism by which the network gradually gets better.

A little more detail on two specific choices in this code: BCEWithLogitsLoss (Binary Cross-Entropy with Logits) is used here rather than a standard multi-class loss because this is a multi-label problem — a single chest X-ray can simultaneously show several findings at once (e.g. both "Cardiomegaly" and "Pleural Effusion"), or none at all, rather than belonging to exactly one mutually-exclusive category. This loss treats each of the 14 findings as its own independent yes/no (binary) decision, which is exactly the right framing for that kind of label structure. Adam is used as the optimizer because it's a robust, well-tested default for training neural networks that generally requires little manual tuning to get reasonable results — a very sensible starting choice before you consider experimenting with other optimizers or learning-rate schedules.

Line	What it does (in plain words)
BCEWithLogitsLoss()	The loss function for multi-label classification — treats each finding as its own yes/no prediction
optimizer.zero_grad()	PyTorch accumulates gradients by default, so you must clear them before each new batch
loss.backward()	Computes how much each weight contributed to the error (backpropagation)
optimizer.step()	Actually updates the weights using those gradients
.to(device)	Moves data/model to GPU if available — image models train far faster on GPU

Finally, notice model.train() at the top of the loop — this switches certain layers (like Dropout, which randomly "turns off" some neurons during training as a regularization technique, and BatchNorm, which normalizes activations using batch statistics) into their training-time behaviour. You'll see its counterpart, model.eval(), used later whenever you evaluate or deploy the model, which switches those same layers into their consistent, non-random inference-time behaviour instead — forgetting to switch between these two modes at the right time is a very common and easy-to-miss source of subtly wrong results in PyTorch projects.

> 💡 *Try it yourself: run 1 epoch on a small subset (e.g. 200 images) first, just to confirm the loop runs end-to-end without errors, before committing to a full multi-hour training run.*

Module 5 — Clinical Text with Transformers

5.1 From TF-IDF to contextual embeddings

If you've worked with text data before in a tabular-adjacent way, you've probably used something like TF-IDF or a simple bag-of-words representation: each document becomes a row of numbers, one column per vocabulary word, where the value reflects how important or frequent that word is in that document. This works reasonably well for many classic text-classification tasks, but it has a significant blind spot — it treats each word as an isolated, context-free unit. The word "opacity" gets exactly the same numeric representation whether it appears in "lung opacity consistent with pneumonia" or in a sentence about window glass — TF-IDF has no way to know these are different senses of the word, because it never looks at surrounding context at all.

A transformer-based language model like BERT (Bidirectional Encoder Representations from Transformers) was specifically designed to fix exactly this limitation. Instead of representing each word with one fixed vector regardless of context, a transformer processes an entire sentence at once and produces a vector for each word that has been informed by every other word around it (this is what "bidirectional" refers to — it looks both left and right, not just at preceding words as some earlier models did). Under the hood it does this using a mechanism called self-attention, which lets every word in the sentence "look at" every other word and decide how much each of them should influence its own representation — you'll come back to attention directly in Module 7 when visualising it for explainability. The practical result you need for this project is simpler than the underlying mechanism: a transformer gives you word (and whole-sentence) representations that actually reflect meaning-in-context, which is a substantially richer signal than TF-IDF for a nuanced domain like radiology reports, where the exact same words can describe very different clinical pictures depending on their surrounding phrasing ("no evidence of" completely flips the meaning of whatever follows it, for instance).

For clinical and biomedical text specifically, it's worth using a version of BERT that has been further pretrained (or "fine-tuned" at the pretraining stage) on medical text rather than general web text — Bio_ClinicalBERT, used below, was pretrained on a large collection of real clinical notes, so it already has a head start on medical vocabulary, abbreviations, and phrasing patterns that a general-purpose BERT model trained only on Wikipedia and books would find unfamiliar.

5.2 Tokenizing and encoding a report

Before any of this can happen, raw text needs to be converted into numbers a model can process — this step is called tokenization, and it's the text-data equivalent of the image preprocessing pipeline you built in Module 2. A tokenizer splits raw text into smaller pieces (tokens — these are often whole words, but can also be sub-word fragments for rarer or more complex words) and converts each piece into a numeric ID by looking it up in a fixed vocabulary the model was trained with.

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained('emilyalsentzer/Bio_ClinicalBERT')
text_model = AutoModel.from_pretrained('emilyalsentzer/Bio_ClinicalBERT')

report = 'The cardiac silhouette is normal. No focal consolidation.'

tokens = tokenizer(
```

```
    report,
    padding='max_length',
    truncation=True,
    max_length=128,
    return_tensors='pt'
)
# tokens['input_ids']      -> word IDs, shape [1, 128]
# tokens['attention_mask'] -> 1 for real tokens, 0 for padding

with torch.no_grad():
    output = text_model(**tokens)

cls_embedding = output.last_hidden_state[:, 0, :]   # shape [1, 768]
# this single 768-number vector summarizes the whole report
```

A few of these arguments deserve unpacking, since they'll reappear every time you touch a transformer model. padding='max_length' and max_length=128 exist because, just like the fixed image size from Module 2, a transformer needs every input in a batch to have the same sequence length. Since reports vary naturally in length, shorter ones are padded out with a special, meaningless filler token up to the chosen maximum length, while longer ones get truncated (cut off) at that same length — so max_length is a genuine design decision: too short and you lose information from long reports, too long and you waste computation on padding for short ones. The attention_mask that comes back alongside input_ids is how the model is told which of those 128 positions are real, meaningful tokens (marked 1) versus which are just padding filler (marked 0), so the padding itself never influences the model's actual understanding of the sentence.

torch.no_grad() wraps the call to the text model here because at this point you're only doing inference — running text through an already-trained model to get its output — not training the model itself. Gradient tracking (which PyTorch does automatically by default, in order to later run backpropagation) has a real memory and computation cost, so explicitly disabling it whenever you don't need it, as here, is a small but genuinely useful efficiency habit.

Finally, last_hidden_state[:, 0, :] extracts one particular vector out of the model's full output. A transformer actually outputs one vector per token (so, up to 128 vectors for this example, matching max_length), each one a contextualized representation of that specific token. By a convention BERT-style models are trained to follow, a special token, always placed first in the sequence and called [CLS] (short for "classification"), ends up with a final vector that can be treated as a summary representation of the entire input sequence — which is exactly why index 0 along that dimension is selected here. This one 768-number vector (768 is simply the hidden size Bio_ClinicalBERT was built with) becomes your entire numeric representation of that report's clinical content, ready to be combined with the image representation in Module 6.

Term	Plain-language meaning
Tokenizer	Splits text into sub-word pieces and converts them to numeric IDs the model understands
padding='max_length'	Pads shorter reports with a filler token so every batch has equal-length sequences
attention_mask	Tells the model which tokens are real vs. just padding, so padding doesn't affect the result
CLS token / [:, 0, :]	By convention the first output vector is treated as a summary of the entire input sequence
torch.no_grad()	Tells PyTorch not to track gradients here — used when you're only doing inference, not training, which saves memory

Try it yourself: run 5 different report sentences through this code and print the shape of cls_embedding each time — it should always be [1, 768] regardless of sentence length, because that's the model's fixed "summary vector" size.

Module 6 — Fusing Image and Text

You now have two separate, working encoders: the image branch from Modules 3–4 turns a chest X-ray into a numeric vector (1024 numbers, for DenseNet121), and the text branch from Module 5 turns a radiology report

into a different numeric vector (768 numbers, for Bio_ClinicalBERT). Fusion is simply the process of combining those two separate vectors into one shared representation that a final decision-making part of the network can use to make a single, joint prediction — the entire point of building a multimodal model in the first place, rather than two unrelated single-modality models.

There are several established strategies for doing this, ranging from very simple to fairly elaborate (concatenation, weighted/gated combination, and cross-attention are the three you'll most often encounter in the literature), but the simplest and most teachable — and a perfectly reasonable starting point for this project — is concatenation fusion: literally placing the two vectors side by side to form one longer vector, then letting a small additional neural network (often just a couple of fully-connected layers) learn how to weigh and combine the information from both modalities to produce the final prediction.

```
import torch.nn as nn

class MultimodalFusionModel(nn.Module):
    def __init__(self, vision_encoder, text_encoder, num_findings):
        super().__init__()
        self.vision_encoder = vision_encoder   # outputs e.g. 1024-dim vector
        self.text_encoder = text_encoder       # outputs 768-dim vector (Bio_ClinicalBERT)

        fused_dim = 1024 + 768
        self.fusion_head = nn.Sequential(
            nn.Linear(fused_dim, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, num_findings)   # one logit per finding
        )

    def forward(self, image, input_ids, attention_mask):
        img_features = self.vision_encoder(image)                       # [batch, 1024]
        text_out = self.text_encoder(input_ids=input_ids,
                                     attention_mask=attention_mask)
        text_features = text_out.last_hidden_state[:, 0, :]             # [batch, 768]

        fused = torch.cat([img_features, text_features], dim=1)         # [batch, 1792]
        return self.fusion_head(fused)
```

Reading through this class definition end to end: __init__ simply stores references to the two encoders you already built (rather than redefining them from scratch — you're reusing the exact vision and text models from earlier modules), and defines a new fusion_head — a small stack of layers whose only job is to take the combined 1,792-number vector (1024 + 768) and gradually reduce it down to exactly num_findings numbers, one per possible diagnosis/finding. The nn.ReLU() activation and nn.Dropout(0.3) inside it serve the same purposes you'd expect from any standard neural network: ReLU introduces non-linearity so the network can learn more than just a straight-line combination of its inputs, and Dropout randomly disables 30% of neurons during training as a regularization technique to reduce overfitting, the same general idea as regularization techniques (like L1/L2 penalties) you may already know from tabular models.

The forward method is where fusion actually happens, and it's worth being very explicit about what torch.cat([img_features, text_features], dim=1) does: it takes the image branch's 1024-number output and the text branch's 768-number output, for every example in the batch, and literally glues them end-to-end into one 1792-number vector. dim=1 specifies which dimension of the tensor to concatenate along — dimension 0 here is the batch dimension (how many examples are in this batch), so specifying dim=1 ensures you're extending each individual example's feature vector, not accidentally stacking whole examples on top of each other. That single line is, quite literally, the entire fusion mechanism in this simple approach — everything else in the class exists to prepare the two vectors being concatenated, or to process the result afterward.

> 💡 *torch.cat([...], dim=1) is the whole idea of "fusion" here — it just glues the two vectors side by side into one longer vector, and lets a small neural network (fusion_head) learn how to weigh information from each modality. This is the*

simplest fusion strategy; once it works, you can explore cross-attention fusion as a stretch goal (the text branch "looks at" specific parts of the image vector and vice versa).

It's worth being explicit here about why cross-attention fusion is a meaningfully more advanced approach, so you know what you'd be reaching for if you take on that stretch goal later. Concatenation fusion treats the two vectors as fixed, already-finished summaries — the image branch has already condensed the entire X-ray down to one vector before the text ever "sees" it, and vice versa. Cross-attention fusion instead lets the two modalities interact earlier and more richly: specific words in the report can learn to focus on ("attend to") specific regions or feature channels within the image representation, and vice versa, rather than the two modalities only being combined right at the very end. It's a more powerful and more literature-standard technique for state-of-the-art multimodal models, but also meaningfully more complex to implement correctly — which is exactly why this guide recommends concatenation fusion as the version to actually get working first.

Remember to run the unimodal (image-only, text-only) baselines from Modules 4–5 first and record their scores — that's how you'll later prove, with numbers, that fusion actually helped.

Module 7 — Explainability

A model that outputs "87% probability of Pneumonia" with no further justification is, from a clinician's point of view, a black box — and clinicians are (rightly) reluctant to act on a black box's output for something as consequential as a diagnosis. Explainability techniques exist to open that box at least partway: to show, in a human-interpretable way, which parts of the input the model actually relied on to reach its conclusion. This module covers one technique for each modality — Grad-CAM for images, and attention visualization for text.

7.1 Grad-CAM for the image branch

Grad-CAM (Gradient-weighted Class Activation Mapping) answers a very specific and useful question: "for this particular prediction, which regions of the input image mattered most?" It works by looking at the gradients flowing backward from the final prediction into one of the CNN's last convolutional layers (recall from Module 3 that these later layers hold increasingly abstract, task-specific spatial features). A large gradient at a particular spatial location means that changing the activation at that location would meaningfully change the prediction — in other words, the model is paying close attention to that region. Grad-CAM turns this gradient information into a heatmap the same size as the image, which you can overlay directly on top of the original X-ray to visually highlight the regions that most influenced the model's decision.

```
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
from pytorch_grad_cam.utils.image import show_cam_on_image
import numpy as np

target_layer = [model.features.denseblock4]   # last conv block
cam = GradCAM(model=model, target_layers=target_layer)

target_class = 3   # e.g. index for 'Pneumonia' in your label list
grayscale_cam = cam(input_tensor=tensor.unsqueeze(0),
                    targets=[ClassifierOutputTarget(target_class)])[0]

rgb_img = np.array(img_rgb.resize((224,224))) / 255.0
visualization = show_cam_on_image(rgb_img, grayscale_cam, use_rgb=True)
plt.imshow(visualization)
plt.axis('off')
plt.show()
```

The target_layer chosen here — the last dense block of DenseNet121 — is deliberately the final convolutional block before the network flattens everything down into its classification layer, because that's where the network holds its richest, most semantically meaningful spatial features (early layers hold only generic edges and textures, which produce far less interpretable heatmaps). target_class lets you generate a separate explanation for each

specific finding — "why did the model think this was Pneumonia" is a genuinely different question, with a potentially different answer, from "why did the model think this was Cardiomegaly," even on the exact same input image, since different findings may be driven by different visual evidence in different parts of the X-ray.

As a sanity check once you have this working: for an obvious, textbook-clear case of a given finding, the resulting heatmap should highlight a clinically plausible region (for a lung-related finding, that generally means somewhere within the lung fields, not, say, concentrated entirely on a corner of the image containing no anatomy at all). If your heatmaps are consistently highlighting irrelevant regions — image borders, text annotations burned into the scan, or areas with no anatomical relevance — this is often a red flag that the model has latched onto a spurious shortcut in the training data (for example, a hospital's specific scanner watermark that happens to correlate with a particular label) rather than genuine clinical evidence, and is well worth investigating before you trust the model's predictions further.

7.2 Attention weights for the text branch

Recall from Module 5 that transformers use a self-attention mechanism internally, where every token learns how much to "attend to" every other token when building its contextual representation. Conveniently, you don't need any extra library for text explainability the way you did for images — HuggingFace's transformer models can simply be asked to also return their internal attention scores alongside their normal output, which you can then inspect directly.

```
output = text_model(**tokens, output_attentions=True)
attentions = output.attentions   # tuple of attention tensors, one per layer
last_layer_attn = attentions[-1][0].mean(dim=0)   # average across attention heads
# last_layer_attn[i][j] roughly = how much token i 'attended to' token j
```

output_attentions=True simply switches on the return of these internal scores, which are normally discarded since most use cases don't need them. attentions comes back as one entry per transformer layer (Bio_ClinicalBERT, like most BERT-family models, has several stacked transformer layers, each with its own attention scores), and within each layer there are multiple attention heads — separate, parallel attention computations that the model learns to specialize differently (one head might specialize in tracking negation words like "no" or "without," another in tracking anatomical terms, and so on, though this specialization emerges from training rather than being explicitly programmed). Averaging across heads with .mean(dim=0), as done here, gives a single simplified view of overall attention for a given layer, which is usually sufficient for a quick visual sanity check of which words the model is focusing on, even though a full analysis could look at individual heads separately.

Module 8 — Evaluating a Medical AI Model Correctly

Evaluation is where a lot of otherwise-solid medical AI projects go quietly wrong, so it deserves careful attention rather than being treated as an afterthought once training is done. The core problem is that medical finding labels are almost always heavily imbalanced — the overwhelming majority of chest X-rays are normal with respect to any single specific rare finding. If, say, only 5% of your X-rays are positive for a particular finding, a model that has learned nothing at all and simply predicts "negative" for every single image would still score 95% accuracy — a number that looks impressive but reflects zero actual diagnostic skill. This is exactly the same imbalanced-classification trap you'd already be alert to in tabular fraud-detection or rare-disease-prediction problems; it doesn't go away just because the input is now an image.

```
from sklearn.metrics import roc_auc_score, f1_score, precision_recall_fscore_support

# y_true, y_pred_probs: shape [n_samples, n_findings]
auroc_per_finding = roc_auc_score(y_true, y_pred_probs, average=None)
auroc_mean = roc_auc_score(y_true, y_pred_probs, average='macro')

y_pred_binary = (y_pred_probs > 0.5).astype(int)
precision, recall, f1, support = precision_recall_fscore_support(
    y_true, y_pred_binary, average=None
)
```

AUROC (Area Under the Receiver Operating Characteristic curve) is used here specifically because it's threshold-independent — rather than committing to one fixed cutoff (like "probability above 0.5 counts as positive") and measuring accuracy at just that one point, AUROC summarizes how well the model ranks true positive cases above true negative cases across every possible threshold simultaneously. A score of 1.0 means perfect ranking (every positive case gets a higher predicted probability than every negative case); 0.5 means the model's ranking is no better than random guessing. This makes AUROC especially well-suited to imbalanced problems, since it isn't distorted by the base rate of positive cases the way raw accuracy is.

average=None returns a separate score for each of the 14 findings individually, which matters because a single averaged number can hide the fact that a model might be excellent at detecting common findings while being essentially useless at detecting a rare but clinically important one — exactly the kind of failure that a single overall accuracy number would completely mask. average='macro' then computes one overall summary number, but by giving every finding equal weight regardless of how common it is in the dataset, rather than letting common findings dominate the average simply because they have more examples (which is what a "micro" average would effectively do). Precision and recall, computed here via precision_recall_fscore_support (a single sklearn function that conveniently returns all three related metrics plus the support, i.e. how many true examples of each class exist, in one call), should already be familiar from tabular classification work — what's worth flagging specifically for healthcare is which of the two typically matters more.

Metric	Why it matters here
AUROC (per finding)	Threshold-independent; tells you how well the model ranks positive vs. negative cases for that specific finding
Precision / Recall	In healthcare, recall (catching real cases) is often prioritized over precision — a missed diagnosis is usually worse than a false alarm
Macro average	Averages metrics equally across all findings, so rare findings aren't drowned out by common ones
Calibration (reliability diagram)	Checks whether a predicted "80% confidence" actually corresponds to being right ~80% of the time — critical for a tool clinicians will trust

The last row of that table — calibration — is a concept that's easy to overlook coming from a purely tabular-modelling background, but becomes especially important for any tool meant to support real clinical decisions. A model can have excellent AUROC (it correctly ranks positive cases above negative ones) while still being poorly calibrated (the actual numeric probability values it outputs don't mean what they claim to mean — e.g. among all the cases where it predicted "90% confidence," only 60% actually turn out to be positive). A reliability diagram checks exactly this, by grouping predictions into confidence buckets (e.g. 0–10%, 10–20%, and so on) and plotting each bucket's actual observed positive rate against its average predicted probability — a perfectly calibrated model would trace a straight diagonal line. For a clinical decision-support tool specifically, calibration matters just as much as raw discriminative accuracy: a clinician relying on your model's stated confidence level to help decide how urgently to follow up on a finding needs that confidence number to be trustworthy and honest, not just a same-relative-ranking output that happens to also be labelled as a percentage.

Module 9 — Serving Your Model

Once you have a trained model you're satisfied with, the final piece of the project is making it usable outside of a notebook — accessible to a dashboard, a demo, or simply to yourself testing it interactively. This module wraps the model in a small web API using FastAPI, a modern, widely-used Python web framework, chosen here specifically because it's lightweight to set up and automatically generates interactive API documentation for you (visible at the /docs URL shown below) with almost no extra effort on your part.

```
from fastapi import FastAPI, UploadFile, Form
from PIL import Image
import io, torch
```

```
app = FastAPI()
model.eval()   # inference mode: disables dropout etc.

@app.post('/predict')
async def predict(image: UploadFile, report_text: str = Form(...)):
    img_bytes = await image.read()
    img = Image.open(io.BytesIO(img_bytes)).convert('RGB')
    img_tensor = transform(img).unsqueeze(0)   # add batch dimension

    tokens = tokenizer(report_text, padding='max_length',
                       truncation=True, max_length=128, return_tensors='pt')

    with torch.no_grad():
        logits = model(img_tensor, tokens['input_ids'], tokens['attention_mask'])
        probs = torch.sigmoid(logits).squeeze().tolist()

    return {'findings': dict(zip(FINDING_NAMES, probs))}
```

A short walkthrough of what's happening here, piece by piece: model.eval() is the counterpart to model.train() from Module 4, and it's essential to call before serving predictions — it switches Dropout and BatchNorm layers into their consistent, deterministic inference behaviour, so the exact same input reliably produces the exact same output every time, rather than the randomized behaviour those layers exhibit during training. The @app.post('/predict') line is a decorator (a Python feature that wraps a function to add extra behaviour) that tells FastAPI "whenever a POST request arrives at the /predict URL, run this function to handle it" — you don't need to write any of the underlying web-server plumbing yourself; FastAPI handles that entirely.

Inside the function, the incoming image arrives as an UploadFile — essentially raw uploaded file bytes — which needs to be read, wrapped in an in-memory byte stream (io.BytesIO), and opened with PIL exactly as you did with files loaded from disk back in Module 2. Note that .unsqueeze(0) is called on the image tensor: your model, and every model built with the Dataset/DataLoader pattern, always expects a batch dimension first (recall the shapes like [batch, 1024] from Module 6), even when you're only sending in a single image for a single real-time prediction — unsqueeze(0) simply adds a batch dimension of size 1 so the tensor's shape matches what the model expects. Finally, torch.sigmoid(logits) converts the model's raw output scores (logits — unbounded numbers that can be positive or negative) into proper probabilities between 0 and 1 for each finding, which is the natural counterpart to BCEWithLogitsLoss from Module 4 (that loss function actually applies this same sigmoid internally when computing the training loss, which is why the raw model output during training is called a "logit" rather than a probability — the API layer needs to apply that sigmoid explicitly itself, since the loss function no longer does it for you at inference time).

Run it with: uvicorn app:app --reload — then test at http://localhost:8000/docs, which FastAPI generates automatically.

Module 10 — Glossary & Function Cheat-Sheet

Use this table as a fast lookup while you're building, rather than something to memorize up front — most of these terms will feel far more concrete once you've actually typed and run the surrounding code in the modules above at least once.

Term / Function	What it is, in one line
Tensor	PyTorch's array type — like a numpy array, but can track gradients and run on GPU
nn.Module	Base class for any model/layer you build in PyTorch
forward()	The method that defines what happens when you call model(input)
DataLoader	Iterates over your Dataset in shuffled batches
optimizer.zero_grad() / .step()	Clears / applies gradient updates each training step

loss.backward()	Runs backpropagation to compute gradients
model.train() / model.eval()	Switches layers like Dropout/BatchNorm between training and inference behavior
torch.no_grad()	Disables gradient tracking — use during inference/evaluation to save memory
transforms.Compose	Chains image preprocessing steps together, applied in order
AutoTokenizer / AutoModel	HuggingFace helpers that auto-load the right tokenizer/model class from a model name
input_ids / attention_mask	The tokenizer's numeric encoding of text, plus a mask marking real vs. padding tokens
torch.cat([...], dim=1)	Concatenates tensors along a chosen dimension — used here to fuse modalities
BCEWithLogitsLoss	Loss function for multi-label (yes/no per class) classification
GradCAM	Explainability tool that highlights image regions driving a prediction
roc_auc_score	sklearn metric measuring ranking quality of predicted probabilities
FastAPI / @app.post	Lightweight Python web framework; @app.post defines an endpoint that accepts POST requests

How to Apply Any New Function You Encounter

No single guide, including this one, can cover every function or edge case you'll run into while building this project — new library versions ship, you'll want to try an architecture variant that isn't documented above, or you'll hit an error message that doesn't match anything written here. The goal of this section is to make sure you're equipped to figure those situations out independently, rather than getting stuck the moment you step outside these pages. When you hit a function or class you haven't seen before anywhere in this project (or beyond it), use this 4-step habit:

1. Check the official docs first — PyTorch (pytorch.org/docs), torchvision, HuggingFace Transformers (huggingface.co/docs), and scikit-learn all have searchable, example-rich documentation. Search "<function name> pytorch docs".
2. Read the signature and defaults. In Python, run help(function_name) or Shift+Tab in Jupyter to see required vs. optional arguments without leaving your notebook.
3. Test it in isolation on a tiny example before wiring it into the full pipeline — e.g. run a tokenizer on one sentence and print the output shape, rather than debugging it inside a 50-line training loop.
4. Print shapes constantly. print(tensor.shape) after every new operation is the single most useful debugging habit in deep learning — almost every bug is a shape mismatch.

Of these four, the habit of testing in isolation on a tiny, throwaway example before integrating anything new into your real pipeline is probably the single highest-leverage one — it turns a potentially confusing bug buried deep inside a long training loop into a small, easy-to-reason-about experiment you can inspect line by line. Treat every unfamiliar function this way, and the size of this guide stops being a limitation, since you'll have a reliable process for extending past it yourself.

Suggested Order to Actually Work Through This

Finally, a concrete suggested order for actually working through everything above, since reading a document front to back is different from building a project incrementally. This mirrors the roadmap phases from the earlier project-blueprint document, but framed here specifically around when each concept in this guide becomes relevant.

1. Module 0 → 1 → 2: get comfortable looking at and transforming images. Don't touch modeling yet.
2. Module 3 → 4: build and train the image-only baseline until it runs end-to-end on a small subset.

3. Module 5: build the text-only baseline the same way.
4. Module 6: only once both unimodal baselines work, build the fused model.
5. Module 7 → 8: add explainability and proper evaluation once the fused model trains successfully.
6. Module 9: wrap it in an API as the final, demoable step.

> 💡 *This guide deliberately doesn't hand you a single ready-to-run end-to-end script — the goal is that you assemble and understand each piece yourself. If a code block doesn't run on the first try, that's normal; debugging it is part of learning the tool, not a sign something is wrong with your approach.*